HypArr

Computations with hyperplane arrangements

0.26.07

2 July 2026

Paul Mücksch

Paul Mücksch
Email: paul.muecksch+uni@gmail.com
Homepage: https://paulmuecksch.github.io
Address: Leibniz Universität Hannover
Institut für Algebra, Zahlentheorie und Diskrete Mathematik
Welfengarten 1
30167 Hannover, Germany

Contents

Chapter 1

Introduction

HYPARR is a GAP 4 package for computations with hyperplane arrangements (see [OT92]), oriented matroids (see [BLVS+99]), their combinatorial and topological invariants, in particular geometric lattices, face posets, Milnor fibers and complements.

Chapter 2

Hyperplane arrangements

Let $\mathbb{K}$ be a field. A hyperplane arrangment is a finite set $\mathscr{A} = \{H_1, \ldots, H_n\}$ of hyperplanes in a finite dimensional $\mathbb{K}$-vector space V.

2.1 Construction of arrangements of hyperplanes

2.1.1 HyperplaneArrangement (for IsList)

▷ `HyperplaneArrangement(R)` (operation)

Returns: A hyperplane arrangement

Constructs a hyperplane arrangement from a list `R` of vectors representing defining linear forms of hyperplanes.

Each vector $[a_1, \ldots, a_\ell]$ in `R` corresponds to the hyperplane

$$a_1 x_1 + \cdots + a_\ell x_\ell = 0.$$

The vectors must lie in the same vector space.

Linearly dependent defining forms are removed automatically.

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ] ]
```

2.2 Attributes of arrangements

2.2.1 Dimension (for IsHyperplaneArrangement)

▷ `Dimension(A)` (attribute)

Returns: A non-negative integer.

Returns the dimension of the ambient vector space in which the hyperplane arrangement `A` is defined.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0]]); Dimension(A);
<HyperplaneArrangement: 2 hyperplanes in 3-space>
3
```

2.2.2 Roots (for IsHyperplaneArrangement)

▷ `Roots(A)` (attribute)

Returns: A list of vectors.

Returns the list of vectors giving defining linear forms (also called *roots*) of the hyperplanes in the arrangement `A`.

Example

```
gap> A:=AGpql(3,3,2); Roots(A);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
[ [ 1, -E(3) ], [ 1, -E(3)^2 ], [ 1, -1 ] ]
```

2.2.3 HArrDefField (for IsHyperplaneArrangement)

▷ `HArrDefField(A)` (attribute)

Returns: A field

Returns the field of definition of `A`. For further compuations, in characterisic 0 only cyclotomic fields are considered.

Example

```
gap> A:=HyperplaneArrangement(Z(3)^0*[[1,0],[0,1],[1,1]]); HArrDefField(A);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
GF(3)
gap> A:=AGpql(5,5,2); HArrDefField(A);
<HyperplaneArrangement: 5 hyperplanes in 2-space>
CF(5)
```

2.2.4 IntersectionLattice (for IsHyperplaneArrangement)

▷ `IntersectionLattice(A)` (attribute)

Returns: An object of type `IsGeomLattice`.

Computes the intersection lattice of the hyperplane arrangement `A`.

The ground set `L` (see `LGroundSet` (2.4.1)) of the lattice is represented level-by-level. The entry `L[k]` contains all intersections of rank k.

Each lattice element is stored as a list of indices referring to hyperplanes in `Roots(A)`.

For example, the list `[1,3]` represents the intersection of the first and third hyperplane of the arrangement.

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,0],[1,1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A); LGroundSet(L);
<Geometric lattice: 3 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ] ],
  [ [ 1, 2 ], [ 1, 3 ], [ 2, 3 ] ],
  [ [ 1, 2, 3 ] ] ]
```

2.2.5 CharPoly (for IsHyperplaneArrangement)

▷ `CharPoly(A)` (attribute)

Returns: a univariate polynomial with integral coefficients.

Returns $\chi_{\mathscr{A}}$ the *characterisitc polynomial* of the arrangement `A`.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> CharPoly(A);
t^3-6*t^2+11*t-6
```

2.2.6 ExpArr (for IsHyperplaneArrangement)

▷ `ExpArr(A)` (attribute)

Returns: a list of integers, or fail

Returns *exponents* of the arrangement $\mathscr{A}$ if the characteristic polynomial factors over the integers.

Example

```
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> ExpArr(A);
[ 1, 2 ]
```

2.2.7 MSetInvL (for IsHyperplaneArrangement)

▷ `MSetInvL(A)` (attribute)

Returns: An object of type IsGeomLattice.

Computes multiset invariants of the intersection lattice (see `IntersectionLattice` (2.2.4)) of the hyperplane arrangement `A`.

For each level of the lattice, the function records how many intersections occur with a given number of defining hyperplanes.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0,],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> MSetInvL(A);
[ [ [ 1, 4 ] ], [ [ 2, 6 ] ], [ [ 4, 1 ] ] ]
```

2.3 Properties of arrangements

2.3.1 IsReal (for IsHyperplaneArrangement)

▷ `IsReal(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is defined over the reals, i.e., if the entries of the defining linear forms are real.

2.3.2 HArrIsIrreducible (for IsHyperplaneArrangement)

▷ `HArrIsIrreducible(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is irreducible.

2.3.3 HArrIsGeneric (for IsHyperplaneArrangement)

▷ `HArrIsGeneric(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is generic. See also `LIsGeneric` (2.5.3).

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> HArrIsGeneric(A);
false
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> HArrIsGeneric(A);
true
```

2.4 Attributes of geometric lattices

2.4.1 LGroundSet (for IsGeomLattice)

▷ `LGroundSet(L)` (attribute)

Returns: list of lists

Returns the ground set of the geometric lattice `L`.

2.4.2 LAtoms (for IsGeomLattice)

▷ `LAtoms(L)` (attribute)

Returns: list

Returns the set of atoms of `L`.

2.4.3 LRank (for IsGeomLattice)

▷ `LRank(L)` (attribute)

Returns: a non-negative integer

Returns the rank of `L`.

2.4.4 LkFlats (for IsGeomLattice)

▷ `LkFlats(L)` (attribute)

Returns: a function

Returns a function extracting the flats of rank k in `L`.

2.4.5 LCircuits (for IsGeomLattice)

▷ `LCircuits(L)` (attribute)

Returns: a list of lists

Gives the circuits of `L`.

2.4.6 LRankFunction (for IsGeomLattice)

▷ `LRankFunction(L)` (attribute)

Returns: A function

Returns the rank function of the geometric lattice `L`.

2.4.7 LGraph (for IsGeomLattice)

▷ `LGraph(L)` (attribute)

Returns: a graph

Constructs the directed graph of the Hasse diagram of L.

2.4.8 LAutGroup (for IsGeomLattice)

▷ `LAutGroup(L)` (attribute)

Returns: a group

Computes the autmorphism group of L as a subgroup of `Sym(LAtoms(L))`.

2.4.9 LMoebius (for IsGeomLattice)

▷ `LMoebius(L)` (attribute)

Returns: function

Returns the Moebius function of `L`.

2.4.10 LCharPoly (for IsGeomLattice)

▷ `LCharPoly(L)` (attribute)

Returns: function

Computes the characteristic polynomial of `L`.

2.4.11 LBases (for IsGeomLattice)

▷ `LBases(L)` (attribute)

Returns: list of bases

Determines the subsets of `LAtoms(L)` which are bases.

Example

2.5 Properties of geometric lattices

2.5.1 LIsIrreducible (for IsGeomLattice)

▷ `LIsIrreducible(L)` (property)

Returns: true or false.

Determines, if L is irreducible.

2.5.2 LIsBoolean (for IsGeomLattice)

▷ LIsBoolean(*L*) (property)

Returns: true or false.

Determines, if L is a boolean lattice.

Example

```
gap> L:=IntersectionLattice(HyperplaneArrangement([[1,0],[1,1]]));
<Geometric lattice: 2 atoms, rank 2>
gap> LIsBoolean(L);
true
gap> L:=IntersectionLattice(HyperplaneArrangement([[1,0],[0,1],[1,1]]));
<Geometric lattice: 3 atoms, rank 2>
gap> LIsBoolean(L);
false
```

2.5.3 LIsGeneric (for IsGeomLattice)

▷ LIsGeneric(*L*) (property)

Returns: true or false.

Determines, if L is generic, i.e. if L is irreducible and all proper intervals are boolean.

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A);
<Geometric lattice: 4 atoms, rank 3>
gap> LIsGeneric(L);
true
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A);
<Geometric lattice: 5 atoms, rank 3>
gap> LIsGeneric(L);
false
```

2.6 Operations

2.6.1 LBsOfFlat (for IsGeomLattice, IsList)

▷ LBsOfFlat(*L*, *m*) (operation)

Returns: list of bases of m

Computes the bases of *m* in *L*.

Example

2.6.2 LBaseCircuit (for IsGeomLattice,IsList,IsInt)

▷ LBaseCircuit(*L*, *B*, *e*) (operation)

Returns: list

Determines the circuit contained in Union(B,[e]).

Example

2.6.3 LIsIsomorphic (for IsGeomLattice,IsGeomLattice)

▷ LIsIsomorphic(*LA*, *LB*) (operation)

Returns: true or false

Determines if the geometric lattices *LA* and *LB* are isomorphic.

Example

```
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]); B:=HyperplaneArrangement([[1,0],[1,1],[1,2]])
<HyperplaneArrangement: 3 hyperplanes in 2-space>
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> LA:=IntersectionLattice(A); LB:=IntersectionLattice(B);
<Geometric lattice: 3 atoms, rank 2>
<Geometric lattice: 3 atoms, rank 2>
gap> LIsIsomorphic(LA,LB);
true
```

2.6.4 IsLEquiv (for IsHyperplaneArrangement,IsHyperplaneArrangement)

▷ IsLEquiv(*A*, *B*) (operation)

Returns: true or false

Determines if the arrangements *A* and *B* have isomorphic intersection lattices.

Example

```
gap> A:=HyperplaneArrangement(([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,3]]);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> B:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> IsLEquiv(A,B);
false
gap> A:=HyperplaneArrangement(([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,1]]);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> IsLEquiv(A,B);
true
```

2.6.5 Essentialization (for IsHyperplaneArrangement)

▷ Essentialization(*A*) (operation)

Returns: A hyperplane arrangement.

Computes the *essentialization* of the hyperplane arrangement *A*. An arrangement is called essential if the intersection of all its hyperplanes is the origin. If this is not the case, the function restricts the arrangement to a complementary subspace so that the resulting arrangement becomes essential.

Example

```
gap> A:=HyperplaneArrangement(([[1,0,0],[0,1,0],[1,1,0]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> B:=Essentialization(A); Roots(B);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
[ [ 1, 0 ], [ 0, 1 ], [ 1, 1 ] ]
```

2.6.6 HArrRestriction (for IsHyperplaneArrangement, IsInt)

▷ HArrRestriction(*A*, *i*) (operation)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement `A` to the `i`-th hyperplane of `Roots(A)`.

2.6.7 HArrRestriction (for IsHyperplaneArrangement, IsList)

▷ `HArrRestriction(A, S)` (operation)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement `A` to the subspace orthogonal to the vectors in `S`.

2.6.8 HArrAddition (for IsHyperplaneArrangement, IsList)

▷ `HArrAddition(A, h)` (operation)

Returns: A hyperplane arrangement.

Computes the *addition* of the arrangement `A` with respect to hyperplane `h`.

2.6.9 HArrDeletion (for IsHyperplaneArrangement, IsInt)

▷ `HArrDeletion(A, i)` (operation)

Returns: A hyperplane arrangement.

Computes the *deletion* of the arrangement `A` with respect to `i`-th hyperplane of `Roots(A)`.

Chapter 3

Special arrangements

The following describes functions to construct some special arrangements.

3.1 Global methods

3.1.1 AGpql

▷ `AGpql(p, q, l)` (function)

Returns: A hyperplane arrangement.

Constructs the reflection arrangement associated to the monomial complex reflection group $G(p,q,l)$. The hyperplanes are defined by equations of the form

$$x_i = \zeta^k x_j$$

where ζ is a primitive p-th root of unity.

Example

```
gap> A := AGpql(2,1,3);
 <HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ],
  [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

3.1.2 SsS3

▷ `SsS3(n)` (function)

Returns: A hyperplane arrangement.

Generates the irreducible supersolvable simplicial arrangement of rank 3 with n hyperplanes. n must either be even oder congruent 1 mod 4. In any case, $n \geq 6$.

Example

```
gap> A:=SsS3(9); Roots(A);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
[ [ 0, 0, 1 ], [ 0, 1, 0 ],
  [ -1/2*E(8)+1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 0 ],
  [ -1, 0, 0 ],
  [ -1/2*E(8)+1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 0 ],
```

```
  [ 1/2*E(8)-1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 1 ],
  [ -1/2*E(8)+1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 1 ],
  [ -1/2*E(8)+1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 1 ],
  [ 1/2*E(8)-1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 1 ] ]
```

3.1.3 ConnectedSubgraphArr (for IsList)

▷ `ConnectedSubgraphArr(Es)` (operation)

Returns: A hyperplane arrangement.

Generates the connected subgraph arrangement (see [CK22]) of the graph with edges `Es`.

Let $G = (N, E)$ be an undirected graph with vertex set $N = [n]$ and edge set E. For $I \subseteq N$, let $G[I]$ be the induced subgraph of G on the set of vertices I. For $I \subseteq N$, define the hyperplane

$$H_I := \ker \sum_{i \in I} x_i.$$

The connected subgraph arrangement consists of hyperplanes $\{H_I \mid \varnothing \neq I \subseteq N \text{ if } G[I] \text{ is connected } \}$.

Example

```
gap> A:=ConnectedSubgraphArr([[1,2],[1,3],[2,3]]); Roots(A);
<HyperplaneArrangement: 7 hyperplanes in 3-space>
[ [ 1, 1, 1 ], [ 0, 1, 1 ], [ 0, 0, 1 ], [ 0, 1, 0 ],
  [ 1, 0, 1 ], [ 1, 0, 0 ], [ 1, 1, 0 ] ]
```

3.1.4 GraphicArr (for IsList)

▷ `GraphicArr(Es)` (operation)

Returns: A hyperplane arrangement.

Generates the graphic arrangement of the graph with edges `Es`.

Let $G = (V, E)$ be an undirected graph with vertex set $V = [\ell]$ and edge set E. The graphic arrangement has hyperplanes

$$\{\{x_i = x_j\} \mid \{i, j\} \in E\}.$$

Example

Chapter 4

Oriented Matroids

Oriented matroids are abstractions of real hyperplane arrangements. The following describes functions to handle oriented matroids and compute associated cell complexes and invariants. See [BLVS+99].

4.1 Construction of oriented matroids

4.1.1 OrientedMatroid (for IsList)

▷ `OrientedMatroid(R, or, A, or[, CCs, c], or[, r, n, ChiroCoreList])` (operation)

Returns: An oriented matroid OM.

Constructs am oriented matroid from

- a list `R` of vectors representing defining linear forms of real hyperplanes,
- a real hyperplane arrangement `A`,
- a list of lists `CVs` of signed covectors and a string `c="cv"`,
- intergers `n,r` giving the size of the groundset and the rank and a list `ChiroCoreList` $\subseteq \{0,1,-1\}^m (m = \binom{n}{r})$ presenting the chirotope

of the oriented matroid.

Example

```
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(Roots(A));
<OrientedMatroid: 6 elements, rank 3>
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OrientedMatroid([[[1,1],[-1,1],[1,-1],[-1,-1]],[[0,1],[0,-1],[1,0],[-1,0]],[[0,0]]],"cv");
<OrientedMatroid: 2 elements, rank 2>
gap> OrientedMatroid(2,3,[1,1,1]);
<OrientedMatroid: 3 elements, rank 2>
```

4.2 Attributes

4.2.1 OMRank (for IsOrientedMatroid)

▷ `OMRank(OM)` (attribute)

Returns: An integer

Returns the rank of the oriented matroid *OM*.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMRank(OM);
3
```

4.2.2 OMGroundSet (for IsOrientedMatroid)

▷ OMGroundSet(*OM*) (attribute)

Returns: A list

Returns the ground set of the oriented matroid OM.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMGroundSet(OM);
[ 1, 2, 3, 4, 5, 6 ]
```

4.2.3 OMLForms (for IsOrientedMatroid)

▷ OMLForms(*OM*) (attribute)

Returns: A list of linear forms or `fail`

Returns the list of linear forms associated with the oriented matroid OM. If the internal component `OM!.lforms` is not bound, `fail` is returned.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMLForms(O);
[ [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

4.2.4 OMGeomLattice (for IsOrientedMatroid)

▷ OMGeomLattice(*OM*) (attribute)

Returns: An object of type `IsGeomLattice` or `fail`

Constructs the geometric lattice associated with the oriented matroid OM. If OM is linear (as determined by `OMIsLinear(OM)`), the lattice is computed as the intersection lattice of the hyperplane arrangement given by `OMLForms(OM)`, stored in the internal component `OM!.lattice`, and returned.

Example

```
gap> O := OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<OrientedMatroid: 4 elements, rank 3>
gap> L:=OMGeomLattice(O); LGroundSet(L);
<Geometric lattice: 4 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ] ],
  [ [ 1, 2 ], [ 1, 3 ], [ 2, 3 ],
      [ 1, 4 ], [ 2, 4 ], [ 3, 4 ] ],
  [ [ 1, 2, 3, 4 ] ] ]
gap> O := OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> L:=OMGeomLattice(O); LGroundSet(L);
```

```
<Geometric lattice: 4 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ] ],
  [ [ 1, 2 ], [ 1, 3 ], [ 2, 3 ],
      [ 1, 4 ], [ 2, 4 ], [ 3, 4 ] ],
  [ [ 1, 2, 3, 4 ] ] ]
```

4.2.5 OMCocircuits (for IsOrientedMatroid)

▷ `OMCocircuits(`*OM*`)` (attribute)

Returns: A list

Computes the (signed) cocircuits of the oriented matroid OM.

The result is a list of sign vectors.

Example

```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]); OMCocircuits(O);
<OrientedMatroid: 3 elements, rank 2>
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
```

4.2.6 OMCircuits (for IsOrientedMatroid)

▷ `OMCircuits(`*OM*`)` (attribute)

Returns: A list

Computes the (signed) circuits of the oriented matroid OM.

The result is a list of sign vectors.

Example

4.2.7 OMTopes (for IsOrientedMatroid)

▷ `OMTopes(`*OM*`)` (attribute)

Returns: A list

Returns the set of topes, i.e., the maximal covectors of *OM*

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> OMTopes(O);
[ [ -1, 1, -1 ], [ 1, -1, 1 ], [ -1, -1, -1 ],
    [ 1, 1, 1 ], [ -1, 1, 1 ], [ 1, -1, -1 ] ]
```

4.2.8 TopeGraph (for IsOrientedMatroid)

▷ `TopeGraph(`*OM*`)` (attribute)

Returns: A graph

Constructs the tope graph of the oriented matroid *OM*. The topes are obtained as the maximal covectors from `OMCovectors(OM)`. Two topes are connected by an edge if their separating set consists of one element.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> G := ShallowCopy(TopeGraph(O));
gap> Vertices(G);
[ 1 .. 6 ]
gap> IsSimpleGraph(G);
true
gap> UndirectedEdges(G);
[ [ 1, 3 ], [ 1, 5 ], [ 2, 4 ], [ 2, 6 ], [ 3, 6 ], [ 4, 5 ] ]
```

4.2.9 OMCovectors (for IsOrientedMatroid)

▷ OMCovectors(*OM*) (attribute)

Returns: A list of lists

Computes all covectors of the oriented matroid OM. The result is returned as a list of lists, where the k-th inner list is a list of covectors corresponding to faces of dimension $k-1$.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> OMCovectors(O);
[ [ [ -1, 1, -1 ], [ 1, -1, 1 ], [ -1, -1, -1 ],
    [ 1, 1, 1 ], [ -1, 1, 1 ], [ 1, -1, -1 ] ],
  [ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
    [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ],
  [ [ 0, 0, 0 ] ] ]
```

4.2.10 OMChirotope (for IsOrientedMatroid)

▷ OMChirotope(*OM*) (attribute)

Returns: A function

Returns the chirotope given as a function from $\binom{E}{r} \to \{0, \pm 1\}$, where E is the groundset of *OM* and r is the rank of *OM*.

Example

```
gap> O:=OrientedMatroid(2,3,[1,1,1]); c:=OMChirotope(O);
<OrientedMatroid: 3 elements, rank 2>
function( S ) ... end
gap> List(Combinations([1..3],2),x->c(x));
[ 1, 1, 1 ]
```

4.2.11 OMRankFunction (for IsOrientedMatroid)

▷ OMRankFunction(*OM*) (attribute)

Returns: A function

Returns the rank function of the oriented matroid *OM*

4.3 Properties

4.3.1 OMIsLinear (for IsOrientedMatroid)

▷ OMIsLinear(*OM*) (property)

Returns: true or false

Determines whether the oriented matroid *OM* is linear. This is the case if the internal component OM!.lforms is bound.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMIsLinear(OM);
true
gap> O := OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> OMIsLinear(O);
false
```

4.3.2 HasSimpleSimplex (for IsOrientedMatroid)

▷ HasSimpleSimplex(*OM*) (property)

Returns: true or false

Returns whether the oriented matroid *OM* has a simple simplex, i.e. a simplicial Tope T such that the localizations at its vertices are boolean and there is another rank 1 covector not adjacent to T. If Rank(OM) ≥ 3 and this is true then its Salvetti complex can not be $K(\pi,1)$.

Example

```
gap> O:=OrientedMatroid([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1],[1,1,1,1]]);
<OrientedMatroid: 5 elements, rank 4>
gap> HasSimpleSimplex(O);
  - tope: [ -1, -1, -1, -1, -1 ]
  - vertices: [ [ 0, 0, 0, -1, -1 ], [ 0, 0, -1, 0, -1 ],
  [ 0, -1, 0, 0, -1 ], [ -1, 0, 0, 0, -1 ] ]
true
```

4.3.3 HasSimpleTriangle (for IsOrientedMatroid)

▷ HasSimpleTriangle(*OM*) (property)

Returns: true or false.

Checks all rank 3 localizations for a simple simplex, i.e. a simple triangle.

Example

4.4 Operations

4.4.1 IsLEquiv (for IsOrientedMatroid,IsOrientedMatroid)

▷ IsLEquiv(*OM1*, *OM2*) (operation)

Returns: true or false

Determines if the oriented matroids *OM1* and *OM2* have isomorphic geometric lattices.

Example

```
gap> O1:=OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> O2:=OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<OrientedMatroid: 4 elements, rank 3>
gap> IsLEquiv(O1,O2);
true
gap> O2:=OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,0]]);
<OrientedMatroid: 4 elements, rank 3>
gap> IsLEquiv(O1,O2);
false
```

4.4.2 HasSimpleSimplexRk (for IsOrientedMatroid, IsInt)

▷ HasSimpleSimplexRk(*OM*, *k*) (operation)

Returns: true or false

Returns whether the oriented matroid *OM* has a localiazion of rank *k* with a simple Simplex. If this is true for $k \geq 3$ then its Salvetti complex can not be $K(\pi, 1)$.

Example

```
gap> O:=OrientedMatroid([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1],[1,1,1,0]]);
<OrientedMatroid: 5 elements, rank 4>
gap> HasSimpleSimplex(O);
false
gap> HasSimpleSimplexRk(O,3);
true
```

4.4.3 OMLocalizationRk (for IsOrientedMatroid, IsList, IsInt)

▷ OMLocalizationRk(*OM*, *F*, *k*) (operation)

Returns: An oriented matroid

Returns the rank *k* localization of *OM* at the flat *F*.

Example

```
gap> O:=OrientedMatroid([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1],[1,1,1,0]]);
<OrientedMatroid: 5 elements, rank 4>
gap> L:=OMGeomLattice(O);
<Geometric lattice: 5 atoms, rank 4>
gap> LkFlats(L)(3);
[ [ 1, 2, 3, 5 ], [ 1, 2, 4 ], [ 1, 3, 4 ], [ 2, 3, 4 ], [ 1, 4, 5 ], [ 2, 4, 5 ], [ 3, 4, 5 ] ]
gap> OMLocalizationRk(O,[1,2,3,5],3);
<OrientedMatroid: 4 elements, rank 3>
```

4.4.4 OMTConvexClosure (for IsOrientedMatroid, IsList)

▷ OMTConvexClosure(*OM*, *Ts*) (operation)

Returns: A list of topes (sign vectors)

Computes the convex closure of the set of topes *Ts* in *OM*.

Example

4.5 Global methods

4.5.1 OMOperation

▷ `OMOperation(`*sv1, sv2*`)` (function)

Returns: A list

Performs the oriented matroid operation on two signed vectors `sv1` and `sv2`. For each position `i`, if `sv1[i]` is nonzero, it is used; otherwise `sv2[i]` is used. Returns a new signed vector representing the result of the operation.

Example

```
gap> CVs:=OMCovectors(O)[2];
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
gap> sv1 := CVs[3]; sv2 := CVs[5];
[ 1, 0, 1 ]
[ -1, 1, 0 ]
gap> OMOperation(sv1, sv2);
[ 1, 1, 1 ]
gap> OMOperation(sv2, sv1);
[ -1, 1, 1 ]
```

4.5.2 OrderCovec

▷ `OrderCovec(`*arg*`)` (function)

Order function for sign vectors.

4.5.3 IsOMEquiv

▷ `IsOMEquiv(`*OM1, OM2*`)` (function)

Returns: true or false

Determines, if the oriented matroids *OM1* and *OM2* are isomorphic.

4.5.4 SeparatingSet

▷ `SeparatingSet(`*arg*`)` (function)

The separating set of two sign vectors.

Chapter 5

Free arrangements

The following describes functions to analyse freeness properties of arrangements such as *inductive freeness* due to H. Terao [Ter80] and *divisionell freeness* due to T. Abe [Abe16]. Moreover, generators and projective dimension of higher derivation modules can be computed.

5.1 Module of logarithmic vector fields aka derivation modules

The functions in this section all use the SINGULAR package to call functions from SINGULAR [DGPS24] for commutative algebra calculations.

5.1.1 DerModule (for IsHyperplaneArrangement)

▷ `DerModule(A[, mult])` (operation)

Returns: Derivation module

Computes the *(multi) derivation module* of the arrangement `A` (with multiplicities given as a list by `mult`).

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> D:=DerModule(A);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
gap> D:=DerModule(A,[2,1,3,4]);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
```

5.1.2 DerPModule (for IsHyperplaneArrangement, IsInt)

▷ `DerPModule(A, p[, mult])` (operation)

Returns: Derivation module

Computes the *(multi) p-derivation module* $D^p(A)$ ($D^p(A,m)$) of the arrangement `A` (with multiplicities given as a list by `mult`).

Example

5.1.3 DerModGenerators (for IsDerivationModule)

▷ DerModGenerators(*D*) (attribute)

Returns: list

Returns the (minimal) generators of the module *D*.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> D:=DerModule(A);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
gap> DerModGenerators(D);
[ [ x_1, x_2, x_3 ],
  [ 0, -x_2*x_3, x_2*x_3 ],
  [ 0, x_2*x_3, x_1*x_3+x_3^2 ],
  [ 0, x_1*x_2+x_2^2+x_2*x_3, 0 ] ]
```

5.1.4 DerModPRing (for IsDerivationModule)

▷ DerModPRing(*D*) (attribute)

Returns: Polynomial ring

Returns the defining polynomial ring of *D*.

Example

```
gap> D:=DerModule(AGpql(3,3,2)); DerModPRing(D);
<field in characteristic 0>[x_1,x_2]
```

5.1.5 DerModDegreeSequence (for IsDerivationModule)

▷ DerModDegreeSequence(*D*) (attribute)

Returns: list

Returns the sequence of degrees of (minimal) generators of the graded module *D*.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> DerModDegreeSequence(DerModule(A));
[ 1, 2, 2 ]
```

5.1.6 DerModProjDim (for IsDerivationModule)

▷ DerModProjDim(*D*) (attribute)

Returns: A non-negative integer

Computes the projective dimension of the module *D*.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> D:=DerModule(A);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
```

```
gap> DerModProjDim(D);
1
```

5.2 Freeness properties

5.2.1 DerModIsFree (for IsDerivationModule)

▷ DerModIsFree(*D*) (property)

Returns: true or false

Determines, if *D* is a free module over DerModPRing(D).

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> D:=DerModule(A); DerModIsFree(D);
true
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> D:=DerModule(A); DerModIsFree(D);
false
```

5.2.2 HArrIsFree (for IsHyperplaneArrangement)

▷ HArrIsFree(*A*) (property)

Returns: true or false

Determines, if *A* is free, i.e. if the derivation module is a free module over the coordinate ring of the ambient space.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> HArrIsFree(A);
true
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> HArrIsFree(A);
false
```

5.2.3 HArrIsLocallyFree (for IsHyperplaneArrangement)

▷ HArrIsLocallyFree(*A*) (property)

Returns: true or false

Determines, if *A* is locally free, i.e. if all proper localizations are free.

Example

5.2.4 IsInductivelyFree (for IsHyperplaneArrangement)

▷ IsInductivelyFree(*A*) (property)

Returns: true or false.

Determines whether the arrangement *A* is *inductively free*.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> IsInductivelyFree(A);
true
```

5.2.5 IsLocallyInductivelyFree (for IsHyperplaneArrangement)

▷ IsLocallyInductivelyFree(A) (property)

Returns: true or false

Determines, if A is locally free, i.e. if all proper localizations are inductively free.

Example

5.2.6 IsDivisionallyFree (for IsHyperplaneArrangement)

▷ IsDivisionallyFree(A) (property)

Returns: true or false.

Determines whether the arrangement A is *divisionally free*.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> IsDivisionallyFree(A);
true
gap> A:=AGpql(3,3,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> IsDivisionallyFree(A);
false
```

5.2.7 IsLocallyDivisionallyFree (for IsHyperplaneArrangement)

▷ IsLocallyDivisionallyFree(A) (property)

Returns: true or false

Determines, if A is locally free, i.e. if all proper localizations are divisionallys free.

Example

5.2.8 IsMATFree (for IsHyperplaneArrangement)

▷ IsMATFree(A) (property)

Returns: true or false.

Determines whether the arrangement A is *MAT-free*, i.e. whether it can be built up from the empty arrangement by successively adding, at each step, an independent set of hyperplanes as permitted by the *Multiple Addition Theorem* due to T. Abe, M. Barakat, M. Cuntz, T. Hoge and H. Terao [ABC+16].

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> IsMATFree(A);
true
```

5.2.9 MATFreePart (for IsHyperplaneArrangement)

▷ `MATFreePart(A)` (attribute)

Returns: list, or fail

Returns a chain of subsets of `Roots(A)` witnessing that `A` is *MAT-free* (see `IsMATFree` (5.2.8)), i.e. a list $[B_1,\dots,B_k]$ of pairwise disjoint independent sets of hyperplanes partitioning `Roots(A)`, such that at each step the arrangement defined by $B_1\cup\cdots\cup B_i$ arises from the one defined by $B_1\cup\cdots\cup B_{i-1}$ by the Multiple Addition Theorem. Returns `fail` if `A` is not MAT-free.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> MATFreePart(A);
[ [ [ 0, 1, 0 ], [ 1, 1, 0 ], [ 1, 1, 1 ] ], [ [ 1, 0, 0 ], [ 0, 0, 1 ] ] ]
```

Chapter 6

Further properties of arrangements, geometric lattices, and oriented matroids

The following describes functions to analyse further properties of arrangements such as *formality*, *supersolvability*, *simpliciality*, *factoredness*, *inductive factoredness* ...

6.1 Supersolvable arrangements

6.1.1 LIsModularPair (for IsGeomLattice, IsList, IsList)

▷ `LIsModularPair(L, m1, m2)` (operation)

Returns: true or false

Determines if *m1* and *m2* form a modular pair in L.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A);
<Geometric lattice: 6 atoms, rank 3>
gap> LGroundSet(L);
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ], [ 5 ], [ 6 ] ],
  [ [ 1, 2 ], [ 1, 3, 6 ], [ 2, 3, 5 ], [ 1, 4, 5 ],
    [ 2, 4, 6 ], [ 3, 4 ], [ 5, 6 ] ],
  [ [ 1, 2, 3, 4, 5, 6 ] ] ]
gap> m1:=[1,2]; m2:= [5,6]; LIsModularPair(L,m1,m2);
false
gap> m1:=[1,2]; m2:= [2,4,6]; LIsModularPair(L,m1,m2);
true
```

6.1.2 LIsModularFlat (for IsGeomLattice, IsList)

▷ `LIsModularFlat(L, m)` (operation)

Returns: true or false

Determines if *m* is a modular flat in L.

Example

```
gap> A:=AGpql(2,2,3); L:=IntersectionLattice(A); LGroundSet(L);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
```

```
<Geometric lattice: 6 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ], [ 5 ], [ 6 ] ],
 [ [ 1, 2 ], [ 1, 3, 6 ], [ 2, 3, 5 ], [ 1, 4, 5 ],
   [ 2, 4, 6 ], [ 3, 4 ], [ 5, 6 ] ],
  [ [ 1, 2, 3, 4, 5, 6 ] ] ]
gap> m1:=[1,2]; m2:= [2,4,6]; LIsModularFlat(L,m1); LIsModularFlat(L,m2);
false
true
```

6.1.3 LModularFlatsRk (for IsGeomLattice, IsInt)

▷ `LModularFlatsRk(L, k)` (operation)

Returns: list

Determines the modular flats in L of rank k.

Example

```
gap> L:=IntersectionLattice(AGpql(2,1,4));
<Geometric lattice: 16 atoms, rank 4>
gap> LModularFlatsRk(L,3);
[ [ 1, 2, 3, 5, 6, 7, 8, 11, 12 ],
  [ 1, 2, 4, 5, 6, 9, 10, 13, 14 ],
  [ 1, 3, 4, 7, 8, 9, 10, 15, 16 ],
  [ 2, 3, 4, 11, 12, 13, 14, 15, 16 ] ]
```

6.1.4 LIsSupersolvable (for IsGeomLattice)

▷ `LIsSupersolvable(L)` (property)

Returns: true or false.

Determines whether the geometric lattice L is *supersolvable*.

Example

```
gap> L:=IntersectionLattice(AGpql(2,1,4));
<Geometric lattice: 16 atoms, rank 4>
gap> LIsSupersolvable(L);
true
gap> L:=IntersectionLattice(AGpql(2,2,4));
<Geometric lattice: 12 atoms, rank 4>
gap> LIsSupersolvable(L);
false
```

6.1.5 HArrIsSupersolvable (for IsHyperplaneArrangement)

▷ `HArrIsSupersolvable(A)` (property)

Returns: true or false.

Determines whether the arrangement A is *supersolvable*.

Example

```
gap> A:=AGpql(2,1,4);
<HyperplaneArrangement: 16 hyperplanes in 4-space>
gap> HArrIsSupersolvable(A);
true
gap> A:=AGpql(2,2,4);
<HyperplaneArrangement: 12 hyperplanes in 4-space>
```

```
gap> HArrIsSupersolvable(A);
false
```

6.1.6 OMIsSupersolvable (for IsOrientedMatroid)

▷ OMIsSupersolvable(*OM*) (property)

Returns: true or false.

Determines whether the oriented matroid *OM* is *supersolvable*.

Example

```
gap> O:=OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> OMIsSupersolvable(O);
false
gap> C:=List(Combinations(Roots(AGpql(2,2,3)),3),x->pos(Determinant(x)));
[ 1, 1, 1, 1, 0, -1, -1, -1, -1, 0, 1, 0, 1, -1, 0, 1, 1, 1, -1, -1 ]
gap> O:=OrientedMatroid(3,6,C);
<OrientedMatroid: 6 elements, rank 3>
gap> OMIsSupersolvable(O);
true
```

6.2 Formal arrangements

6.2.1 IsFormal (for IsHyperplaneArrangement)

▷ IsFormal(*A*) (property)

Returns: true or false.

Determines whether the arrangement is *formal* in the sense of Falk-Randell [FR87].

Example

```
gap> A1 := Arr([
>     [1,0,0],
>     [0,1,0],
>     [0,0,1],
>     [1,1,1],
>     [2,1,1],
>     [2,3,1],
>     [2,3,4],
>     [3,0,5],
>     [3,4,5]
> ]);
gap>
gap> A2 := Arr([
>     [1,0,0],
>     [0,1,0],
>     [0,0,1],
>     [1,1,1],
>     [2,1,1],
>     [2,3,1],
>     [2,3,4],
>     [1,0,3],
>     [1,2,3]
```

```
> ]);
gap> IsLEquiv(A1,A2);
true
gap> IsFormal(A1);
true
gap> IsFormal(A2);
false
```

6.3 Simplicial arrangements

6.3.1 HArrIsSimplicial (for IsHyperplaneArrangement and IsReal)

▷ `HArrIsSimplicial(A)` (property)

Returns: true or false.

Determines whether the real arrangement `A` is *simplicial*, i.e. if all chambers are simplicial.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1]]); HArrIsSimplicial(A);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
false
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,1]]); HArrIsSimplicia
<HyperplaneArrangement: 6 hyperplanes in 3-space>
true
```

6.4 Falk's weight test

The following still needs to be tested further...

6.4.1 OMBoundedCpx (for IsOrientedMatroid, IsInt)

▷ `OMBoundedCpx(OM, g)` (operation)

Returns: list of list of sign vectors.

Constructs the complex of bounded cells of the affine part of the oriented matroid `OM` with respect to the element `g`.

Example

6.4.2 OMSupportsFalkWeights (for IsOrientedMatroid, IsInt)

▷ `OMSupportsFalkWeights(OM, g)` (operation)

Returns: list or fail

Only for oriented matroids of rank 3. Determines, whether `OM` supports a system of weights as described by M. Falk in [Fal95]. This implies, that the Salvetti complex (see `SalvettiComplex` (7.1.1)) is apsherical. The functionality of the CDDINTERFACE package is used, to determine solutions of a system of linear inequalities.

If the algorithm determines a suitable weight system for `OM`, each list element consists of a weight, and a corner, i.e. a pair of a 2-cell and an adjacent vertex in the bounded complex, given as sign vectors.

Example

```
gap> O:=OrientedMatroid(
>       [[1,1,0],[1,-1,0],[1,0,1],[1,0,-1],[0,1,1],[0,1,-1],[0,0,1]]
> );
<OrientedMatroid: 7 elements, rank 3>
gap> OMIsSupersolvable(O);
false
gap> OMSupportsFalkWeights(O,3);
fail
gap> OMSupportsFalkWeights(O,1);
[ [ 1/2, [ [ 1, 1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 0, 1, 0, 1 ] ] ],
  [ 0, [ [ 1, 1, 1, 1, 1, 1, 1 ], [ 1, 1, 1, 1, 0, 0, 0 ] ] ],
  [ 1/2, [ [ 1, 1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ],
  [ 0, [ [ 1, 1, 1, 1, 1, 1, -1 ], [ 1, 0, 0, 1, 0, 1, -1 ] ] ],
  [ 1/2, [ [ 1, 1, 1, 1, 1, 1, -1 ], [ 1, 1, 1, 1, 0, 0, 0 ] ] ],
  [ 1/2, [ [ 1, 1, 1, 1, 1, 1, -1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ],
  [ 0, [ [ 1, -1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 0, 1, 0, 1 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, 1 ], [ 1, -1, 0, 0, 1, 1, 0 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, -1 ], [ 1, 0, 0, 1, 0, 1, -1 ] ] ],
  [ 0, [ [ 1, -1, 1, 1, 1, 1, -1 ], [ 1, -1, 0, 0, 1, 1, 0 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, -1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ] ]
```

6.5 Factored arrangements

6.5.1 HArrIsFactored (for IsHyperplaneArrangement)

▷ HArrIsFactored(*A*) (attribute)

Returns: A list or fail

Determines if *A* is factored and if so returns some factorization.

Example

```
gap> A:=AGpql(2,1,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> HArrIsFactored(A);
[ [ 1 ], [ 2, 4, 5 ], [ 3, 6, 7, 8, 9 ] ]
gap> A:=AGpql(2,2,4);
<HyperplaneArrangement: 12 hyperplanes in 4-space>
gap> HArrIsFactored(A);
fail
```

6.5.2 HArrFactorizations (for IsHyperplaneArrangement)

▷ HArrFactorizations(*A*) (operation)

Returns: A list

Computes a list of all factorizations of *A* as partitions of [1..|*A*|].

Example

```
gap> A:=AGpql(2,1,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> HArrFactorizations(A);
[ [ [ 1 ], [ 2, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
```

```
  [ [ 1 ], [ 2, 4, 5, 8, 9 ], [ 3, 6, 7 ] ],
  [ [ 2 ], [ 1, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 2 ], [ 1, 4, 5, 6, 7 ], [ 3, 8, 9 ] ],
  [ [ 3 ], [ 1, 6, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 3 ], [ 1, 4, 5, 6, 7 ], [ 2, 8, 9 ] ],
  [ [ 4 ], [ 1, 2, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 5 ], [ 1, 2, 4 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 6 ], [ 1, 3, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 7 ], [ 1, 3, 6 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 8 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 9 ] ],
  [ [ 9 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 8 ] ] ]
```

6.5.3 HArrIsInductivelyFactored (for IsHyperplaneArrangement)

▷ `HArrIsInductivelyFactored(A)` (attribute)

Returns: A list or fail

Determines if A is inductively factored and if so returns some inductive factorization.

Example

```
gap> A:=AGpql(2,1,4);
<HyperplaneArrangement: 16 hyperplanes in 4-space>
gap> HArrIsInductivelyFactored(A);
[ [ [ 1 ], [ 2, 5, 6 ], [ 3, 7, 8, 11, 12 ], [ 4, 9, 10, 13, 14, 15, 16 ] ], true ]
gap> A:=AGpql(2,2,4);
<HyperplaneArrangement: 12 hyperplanes in 4-space>
gap> HArrIsInductivelyFactored(A);
false
```

6.5.4 HArrInductiveFactorizations (for IsHyperplaneArrangement)

▷ `HArrInductiveFactorizations(A)` (operation)

Returns: A list

Computes a list of all inductive factorizations of A as partitions of $[1..|A|]$.

Example

```
gap> A:=AGpql(2,1,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> HArrInductiveFactorizations(A);
[ [ [ 1 ], [ 2, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 1 ], [ 2, 4, 5, 8, 9 ], [ 3, 6, 7 ] ],
  [ [ 2 ], [ 1, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 2 ], [ 1, 4, 5, 6, 7 ], [ 3, 8, 9 ] ],
  [ [ 3 ], [ 1, 6, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 3 ], [ 1, 4, 5, 6, 7 ], [ 2, 8, 9 ] ],
  [ [ 4 ], [ 1, 2, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 5 ], [ 1, 2, 4 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 6 ], [ 1, 3, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 7 ], [ 1, 3, 6 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 8 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 9 ] ],
  [ [ 9 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 8 ] ] ]
```

Chapter 7

Complements of complex arrangements

For a hyperplane arrangement $\mathscr{A}$ in $\mathbb{C}^\ell$, the following provides functions to compute complexes which have the homotopy type of the complement manifold $\mathbb{C}^\ell \setminus \bigcup_{H\in\mathscr{A}} H$.

7.1 Complexes and face posets

7.1.1 SalvettiComplex (for IsOrientedMatroid)

▷ `SalvettiComplex(`*OM, or, A*`)` (attribute)

Returns: A `FacePoset`

Constructs the Salvetti complex [Sal87] associated with the oriented matroid OM or the real hyperplane arrangement *A*.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> SalvettiComplex(A);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

7.1.2 SalvettiComplex (for IsHyperplaneArrangement)

▷ `SalvettiComplex(`*A*`)` (attribute)

See `SalvettiComplex` (7.1.1).

7.1.3 FPGroundSet (for IsFacePoset)

▷ `FPGroundSet(`*FP*`)` (attribute)

Returns: A list

Returns the ground set of the face poset FP.

Example

```
gap> S := SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

```
gap> FPGroundSet(S);
[
  [
    [ [ -1, 1, -1 ], [ -1, 1, -1 ] ],
    [ [ 1, -1, 1 ], [ 1, -1, 1 ] ],
    [ [ -1, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ 1, 1, 1 ], [ 1, 1, 1 ] ],
    [ [ -1, 1, 1 ], [ -1, 1, 1 ] ],
    [ [ 1, -1, -1 ], [ 1, -1, -1 ] ]
  ],
  [
    [ [ -1, 0, -1 ], [ -1, 1, -1 ] ],
    [ [ -1, 1, 0 ], [ -1, 1, -1 ] ],
    [ [ 1, 0, 1 ], [ 1, -1, 1 ] ],
    [ [ 1, -1, 0 ], [ 1, -1, 1 ] ],
    [ [ 0, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ -1, 0, -1 ], [ -1, -1, -1 ] ],
    [ [ 0, 1, 1 ], [ 1, 1, 1 ] ],
    [ [ 1, 0, 1 ], [ 1, 1, 1 ] ],
    [ [ 0, 1, 1 ], [ -1, 1, 1 ] ],
    [ [ -1, 1, 0 ], [ -1, 1, 1 ] ],
    [ [ 0, -1, -1 ], [ 1, -1, -1 ] ],
    [ [ 1, -1, 0 ], [ 1, -1, -1 ] ]
  ],
  [
    [ [ 0, 0, 0 ], [ -1, 1, -1 ] ],
    [ [ 0, 0, 0 ], [ 1, -1, 1 ] ],
    [ [ 0, 0, 0 ], [ -1, -1, -1 ] ],
    [ [ 0, 0, 0 ], [ 1, 1, 1 ] ],
    [ [ 0, 0, 0 ], [ -1, 1, 1 ] ],
    [ [ 0, 0, 0 ], [ 1, -1, -1 ] ]
  ]
]
```

7.1.4 FPOrder (for IsFacePoset)

▷ `FPOrder(`*FP*`)` (attribute)

Returns: A function

Returns the order function of the face poset FP.

7.1.5 BZs1Complex (for IsHyperplaneArrangement)

▷ `BZs1Complex(`*A*`)` (attribute)

Returns: A `FacePoset`

Constructs the complex described by Bjoener and Ziegler in [BZ92] oibtained from the $s^{(1)}$-stratification of a complex hyperplane arrangement *A* which has the homotopy type of the complement.

Example

```
gap> A:=AGpql(3,3,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> Cs1:=BZs1Complex(A);
```

```
<FacePoset of dimension 4 with f-vector [ 194, 1004, 1560, 812, 62 ]>
gap> Cs1CW := FPtoCWCpx(Cs1);
Regular CW-complex of dimension 4
gap> List([0..4],x->Length(Homology(Cs1CW,x)));
[ 1, 9, 24, 16, 0 ]
gap> CharPoly(A);
t^3-9*t^2+24*t-16
```

7.1.6 BZs2Complex (for IsHyperplaneArrangement)

▷ BZs2Complex(*A*) (attribute)

Returns: A FacePoset

Constructs the complex described by Bjoener and Ziegler in [BZ92] oibtained from the $s^{(2)}$-stratification of a complex hyperplane arrangement *A* which has the homotopy type of the complement.

Example

```
gap> A:=AGpql(3,3,2);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> Cs2:=BZs2Complex(A);
<FacePoset of dimension 4 with f-vector [ 52, 132, 96, 16, 0 ]>
```

Be aware that both functions BZs1Complex and BZs2Complex may take a considerable amount of time computing the complex for non-real arrangements since oriented matroid complexes in double dimension and double number of hyperplanes need to be computed. For a real arrangement, the $s^{(1)}$-complex is isomorphic to the Salvetti complex.

7.2 Non-$K(\pi,1)$ arrangements

7.2.1 CCSimpleTriangle (for IsHyperplaneArrangement)

▷ CCSimpleTriangle(*A*) (operation)

Returns: true or false

Tests if there is a convex open subset of the ambient space, such that the arrangement restricted to this convex open subset is a "simple triangle", see also HasSimpleTriangle (4.3.3). This implies that the arrangement is not $K(\pi,1)$.

Example

```
gap> A:=Arr([ [ E(4), E(4), E(4) ], [ 0, E(4), 1 ], [ 1, 0, 1 ], [ 1, 0, 0 ] ]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> CCSimpleTriangle(A);
[ 1, 2, 3 ]
true
```

Chapter 8

Milnor fibers

The Milnor fiber is the typical fiber $f^{-1}(1)$ of the Milnor fibration

$$f : \mathbb{C}^\ell \setminus \cup_{H\in\mathscr{A}} H \to \mathbb{C}^\times, z \mapsto \prod_{H\in\mathscr{A}} \alpha_H(z)$$

of the hyperplane arrangement $\mathscr{A}$, where α_H are defining linear forms.

The following describes functions to construct a regular cell complex having the homotopy type of the Milnor fiber from [MY25] and thus enables computation of homotopy invaraints using e.g. the HAP package [Ell26].

8.1 Complexes

8.1.1 MilnorFiberComplex (for IsOrientedMatroid)

▷ `MilnorFiberComplex(OM)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex of the combinatorial Milnor fiber of the oriented matroid `OM`.

Example

```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> MCpx:=MilnorFiberComplex(O);
<FacePoset of dimension 1 with f-vector [ 3, 6 ]>
```

8.1.2 MilnorFiberComplex (for IsHyperplaneArrangement)

▷ `MilnorFiberComplex(A)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex having the homotopy type of Milnor fiber of the arrangement `A`.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
```

8.2 Functions for (Co)homology computations

8.2.1 FPtoCWCpx (for IsFacePoset)

▷ FPtoCWCpx(*FP*) (operation)

Returns: HAP CW Complex

Converts a HypArr FacePoset into a HAP RegularCWComplex for further computations of topological invariants.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
gap> MCW := FPtoCWCpx(MCpx);
Regular CW-complex of dimension 2
gap> Homology(MCW,0);
[ 0 ]
gap> Homology(MCW,1);
[ 0, 0, 0, 0, 0, 0, 0 ]
```

8.2.2 FPtoSCpx (for IsFacePoset)

▷ FPtoSCpx(*FP*) (operation)

Returns: Simplicial complex

Constructs the order complex of the FacePoset given as the set of maximal simplices, i.e. chains in *FP*.

Example

```
gap> MFP:=MilnorFiberComplex(AGpql(2,2,3));
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
gap> SMF:=FPtoSCpx(MFP);
Simplicial complex of dimension 2.
```

8.2.3 MFModPCohomRing (for IsOrientedMatroid, IsInt)

▷ MFModPCohomRing(*OM*, *p*) (operation)

Returns: HAP mod-p cohomology

Example

8.2.4 MFModPCohomRing (for IsHyperplaneArrangement, IsInt)

▷ MFModPCohomRing(*A*, *p*) (operation)

Returns: HAP mod-p cohomology

Example

Chapter 9

Tope posets

9.1 Operations

9.1.1 TopePoset (for IsOrientedMatroid, IsList)

▷ TopePoset(*OM, BTope*) (operation)

Returns: A TopePoset object

Constructs the tope poset of the oriented matroid *OM* with respect to the base tope *BTope* (given as sign vector).

Example

```
gap> O:=OM([[1,0],[0,1]]); Ts:=OMCovectors(O)[1]; T:=Ts[1];
<OrientedMatroid: 2 elements, rank 2>
[ [ -1, -1 ], [ 1, 1 ], [ -1, 1 ], [ 1, -1 ] ]
[ -1, -1 ]
gap> TP:=TopePoset(O,T);
<TopePoset with 3 topes>
```

9.2 Attributes

9.2.1 TPGroundSet (for IsTopePoset)

▷ TPGroundSet(*TP*) (attribute)

Returns: A list of lists.

The ground set of the tope poset *TP*. It is a list of list, recording the separating elements of the given tope from the base tope.

Example

```
gap> TPGroundSet(TP);
[ [ [  ] ], [ [ 1 ], [ 2 ] ], [ [ 1, 2 ] ] ]
```

9.2.2 TPBaseTope (for IsTopePoset)

▷ TPBaseTope(*TP*) (attribute)

Returns: A sign vector.

Returns the bast tope of the tope poset *TP*.

Example

```
gap> TPBaseTope(TP);
[ -1, -1 ]
```

9.2.3 TPRankPoly (for IsTopePoset)

▷ `TPRankPoly(`*TP*`)` (attribute)

Returns: A polynomial.

Returns the rank generating polynomial of the tope poset *TP*.

Example

```
gap> TPRankPoly(TP);
t^2+2*t+1
```

Chapter 10

Varchenko-Gelfand algebra

10.1 Construction

10.1.1 VGAlgebra (for IsOrientedMatroid, IsField)

▷ `VGAlgebra(`*`OM, field`*`)` (operation)

Returns: Varchenko-Gelfand albgera

Constructs the Varchenko-Gelfand algebra of *OM*.

Example

Chapter 11

Visualization of real 3-arrangements

11.1 Global methods

11.1.1 LaTeXDrawProjPicture

▷ LaTeXDrawProjPicture(*A[, DrawOptions]*) (function)

Returns: A string.

Generates LaTeX tikz-code for a nice projective picture of the real 3-arrangement. If x, y, z are the coordinates, by default, this is the 2-dim affine arrangement obtained by intersecting *A* with the plane $z = 1$.

In the optional parameters *DrawOptions*, a record, the following parameters can be chosen

- *scale*, a positive rational scaling factor,
- *isecps* intersection points are drawn, true or false,
- *Hind* labels for the hyperplanes are added, true or false,
- *deconeH* a vector giving the normal of the plane (=1) with which to intersect *A*,
- *MarkHs* a list of indices of hyperplanes of *A*, these are drawn in another color.

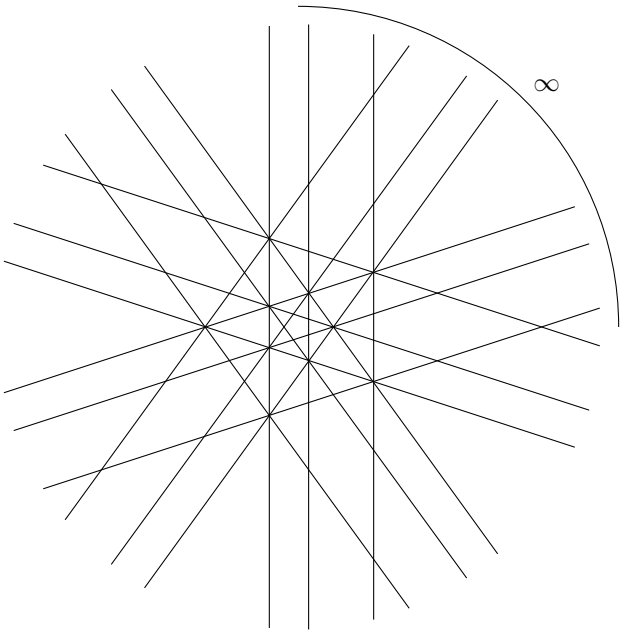
Example

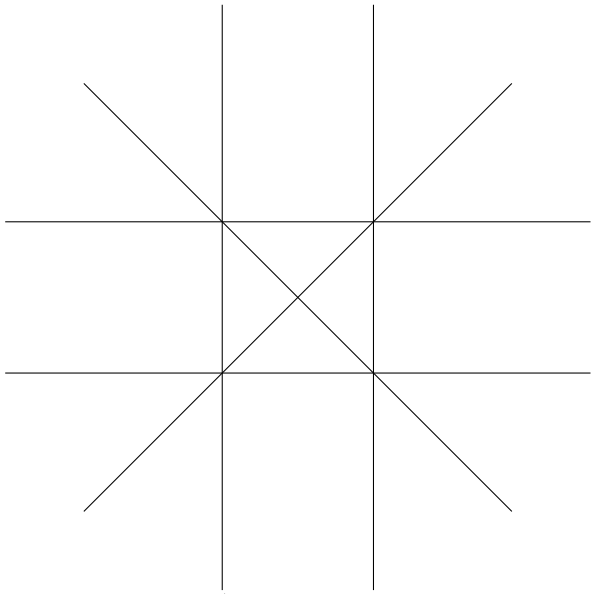
```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> Print(LaTeXDrawProjPicture(A));
\begin{tikzpicture}[scale=1.0]
\draw (-2.83,2.83) -- (2.83,-2.83);  % H_1
\draw (2.83,2.83) -- (-2.83,-2.83);  % H_2
\draw (-1.,3.87) -- (-1.,-3.87);  % H_3
\draw (1.,3.87) -- (1.,-3.87);  % H_4
\draw (3.87,-1.) -- (-3.87,-1.);  % H_5
\draw (3.87,1.) -- (-3.87,1.);  % H_6
\end{tikzpicture}
gap> DrawOpts:=rec(scale:=1/2,isecps:=true,Hind:=true,
>       deconeH:=[1,1,1],MarkHs:=[1,2]);
gap> Print(LaTeXDrawProjPicture(AGpql(2,2,3),DrawOpts));
\begin{tikzpicture}[scale=1.0]
\draw[color=red] (-3.56,1.83) -- (1.83,-3.56);  % H_1
```

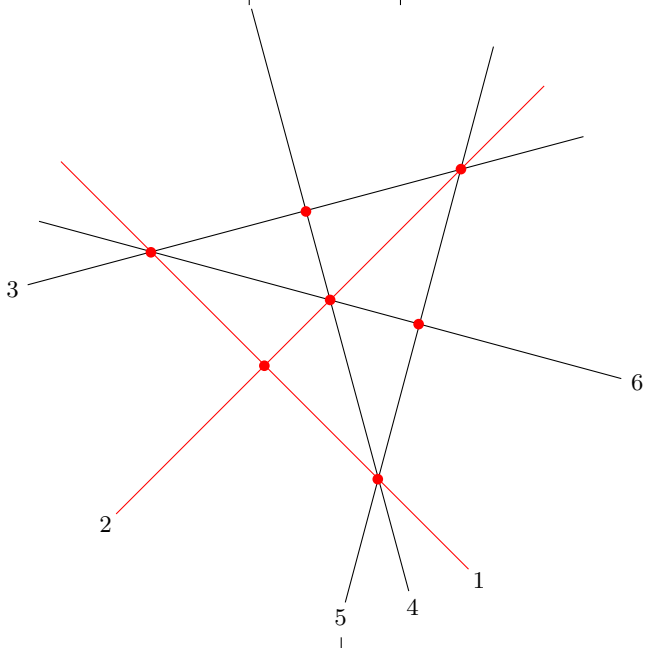
```
\node at (1.97,-3.71) {\small $1$};
\draw[color=red] (2.83,2.83) -- (-2.83,-2.83);  % H_2
\node at (-2.97,-2.97) {\small $2$};
\draw (3.35,2.16) -- (-4.,0.20);  % H_3
\node at (-4.20,0.14) {\small $3$};
\draw (-1.04,3.85) -- (1.04,-3.85);  % H_4
\node at (1.09,-4.06) {\small $4$};
\draw (2.16,3.35) -- (0.20,-4.);  % H_5
\node at (0.14,-4.20) {\small $5$};
\draw (-3.85,1.04) -- (3.85,-1.04);  % H_6
\node at (4.06,-1.09) {\small $6$};

\fill[red] (-0.87,-0.87) circle[radius=2pt];  % P[ 1, 2 ]
\fill[red] (-2.37,0.63) circle[radius=2pt];  % P[ 1, 3, 6 ]
\fill[red] (1.73,1.73) circle[radius=2pt];  % P[ 2, 3, 5 ]
\fill[red] (0.63,-2.37) circle[radius=2pt];  % P[ 1, 4, 5 ]
\fill[red] (0.0,0.0) circle[radius=2pt];  % P[ 2, 4, 6 ]
\fill[red] (-0.32,1.17) circle[radius=2pt];  % P[ 3, 4 ]
\fill[red] (1.17,-0.32) circle[radius=2pt];  % P[ 5, 6 ]
\end{tikzpicture}
```

The preceding example will look as follows


∞




3
6
2
1
5
4

11.1.2 LaTeXDrawSpherePicture

▷ LaTeXDrawSpherePicture(*A*) (function)

Returns: A string.

Generates LaTeX tikz-code for the intersection of a real 3-arrangement with the unit sphere. To compile the LaTeX-code the .sty-file "graphonsphere.sty" (from /doc/LaTeX_Examples) needs to be in the same folder and added via "\usepackage{graphonsphere}".

Example

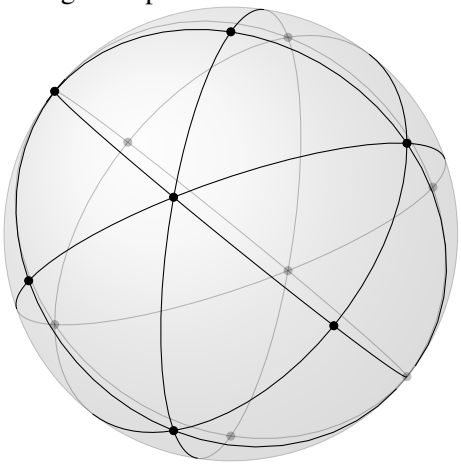
```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> Print(LaTeXDrawSpherePicture(A));
\tdplotsetmaincoords{63}{63}
\begin{tikzpicture}[scale=3,tdplot_main_coords]

\draw[ball color = gray!40, opacity = 0.2] (0,0,0) circle (1cm);

\def\OMcolor{black}
{\color{\OMcolor}
% the vertices of the OM-cpx
\foreach \i/\vx/\vy/\vz in {
1/0./0./1.,
2/-0.57/0.57/0.57,
3/-0.57/-0.57/0.57,
4/0.57/-0.57/0.57,
5/0.57/0.57/0.57,
6/0./1./0.,
7/1./0./0.,
8/0./0./-1.,
9/0.57/-0.57/-0.57,
10/0.57/0.57/-0.57,
11/-0.57/0.57/-0.57,
12/-0.57/-0.57/-0.57,
13/0./-1./0.,
14/-1./0./0.
}{
\node (\i) at (\vx,\vy,\vz) {};
\POnSfb{\vx}{\vy}{\vz}{}
}
%
% the edges of the OM-cpx
\foreach \ax/\ay/\az/\bx/\by/\bz in {
0./0./1./  -0.57/0.57/0.57, %[ 1, 2 ]
0./0./1./  -0.57/-0.57/0.57, %[ 1, 3 ]
0./0./1./  0.57/-0.57/0.57, %[ 1, 4 ]
0./0./1./  0.57/0.57/0.57, %[ 1, 5 ]
-0.57/0.57/0.57/  -0.57/-0.57/0.57, %[ 2, 3 ]
-0.57/0.57/0.57/  0.57/0.57/0.57, %[ 2, 5 ]
-0.57/0.57/0.57/  0./1./0., %[ 2, 6 ]
-0.57/0.57/0.57/  -0.57/0.57/-0.57, %[ 2, 11 ]
-0.57/0.57/0.57/  -1./0./0., %[ 2, 14 ]
-0.57/-0.57/0.57/  0.57/-0.57/0.57, %[ 3, 4 ]
-0.57/-0.57/0.57/  -0.57/-0.57/-0.57, %[ 3, 12 ]
```

```
-0.57/-0.57/0.57/  0./-1./0., %[ 3, 13 ]
-0.57/-0.57/0.57/  -1./0./0., %[ 3, 14 ]
0.57/-0.57/0.57/  0.57/0.57/0.57, %[ 4, 5 ]
0.57/-0.57/0.57/  1./0./0., %[ 4, 7 ]
0.57/-0.57/0.57/  0.57/-0.57/-0.57, %[ 4, 9 ]
0.57/-0.57/0.57/  0./-1./0., %[ 4, 13 ]
0.57/0.57/0.57/  0./1./0., %[ 5, 6 ]
0.57/0.57/0.57/  1./0./0., %[ 5, 7 ]
0.57/0.57/0.57/  0.57/0.57/-0.57, %[ 5, 10 ]
0./1./0./  0.57/0.57/-0.57, %[ 6, 10 ]
0./1./0./  -0.57/0.57/-0.57, %[ 6, 11 ]
1./0./0./  0.57/-0.57/-0.57, %[ 7, 9 ]
1./0./0./  0.57/0.57/-0.57, %[ 7, 10 ]
0./0./-1./  0.57/-0.57/-0.57, %[ 8, 9 ]
0./0./-1./  0.57/0.57/-0.57, %[ 8, 10 ]
0./0./-1./  -0.57/0.57/-0.57, %[ 8, 11 ]
0./0./-1./  -0.57/-0.57/-0.57, %[ 8, 12 ]
0.57/-0.57/-0.57/  0.57/0.57/-0.57, %[ 9, 10 ]
0.57/-0.57/-0.57/  -0.57/-0.57/-0.57, %[ 9, 12 ]
0.57/-0.57/-0.57/  0./-1./0., %[ 9, 13 ]
0.57/0.57/-0.57/  -0.57/0.57/-0.57, %[ 10, 11 ]
-0.57/0.57/-0.57/  -0.57/-0.57/-0.57, %[ 11, 12 ]
-0.57/0.57/-0.57/  -1./0./0., %[ 11, 14 ]
-0.57/-0.57/-0.57/  0./-1./0., %[ 12, 13 ]
-0.57/-0.57/-0.57/  -1./0./0. %[ 12, 14 ]
}{
\GCArcABfb{\ax}{\ay}{\az}{\bx}{\by}{\bz}{color=\OMcolor}
}
}
\end{tikzpicture}
```

The preceding example will look as follows



Sphere picture produced by `LaTeXDrawSpherePicture`.

11.1.3 LaTeXDrawTopeGraph

▷ LaTeXDrawTopeGraph(*A*) (function)

Returns: A string.

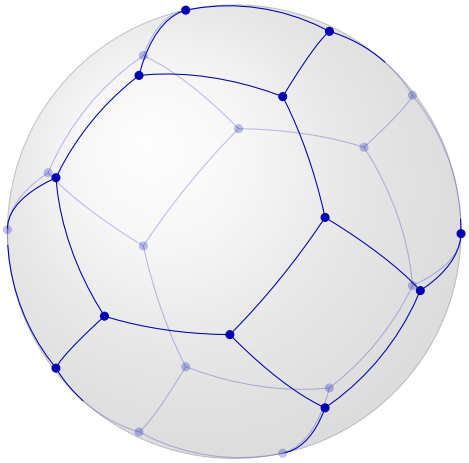
Generates LaTeX tikz-code for the tope graph of a real 3-arrangement on the unit sphere. To compile the LaTeX-code the .sty-file "graphonsphere.sty" (from /doc/LaTeX_Examples) needs to be in the same folder and added via "\usepackage{graphonsphere}".

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> Print(LaTeXDrawTopeGraph(A));
\tdplotsetmaincoords{63}{63}
\begin{tikzpicture}[scale=3,tdplot_main_coords]
\draw[ball color = gray!40, opacity = 0.2] (0,0,0) circle (1cm);
\def\OMcolor{blue!70!black}
{\color{\OMcolor}
% the vertices of the OM-cpx
\foreach \i/\vx/\vy/\vz in {
1/-0.88/-0.47/0.,
2/0.88/0.47/0.,
3/-0.88/0./-0.47,
4/0.88/0./0.47,
5/-0.88/0./0.47,
6/0.88/0./-0.47,
7/-0.88/0.47/0.,
8/0.88/-0.47/0.,
9/-0.47/0./-0.88,
10/0.47/0./0.88,
11/-0.47/0./0.88,
12/0.47/0./-0.88,
13/-0.47/-0.88/0.,
14/0.47/0.88/0.,
15/0./-0.47/-0.88,
16/0./0.47/0.88,
17/0./-0.47/0.88,
18/0./0.47/-0.88,
19/0./-0.88/-0.47,
20/0./0.88/0.47,
21/0./-0.88/0.47,
22/0./0.88/-0.47,
23/0.47/-0.88/0.,
24/-0.47/0.88/0.
}{
\node (\i) at (\vx,\vy,\vz) {};
\POnSfb{\vx}{\vy}{\vz}{}
}
%
% the edges of the OM-cpx
\foreach \ax/\ay/\az/\bx/\by/\bz in {
-0.88/-0.47/0./  -0.88/0./-0.47, %[ 1, 3 ]
-0.88/-0.47/0./  -0.88/0./0.47, %[ 1, 5 ]
-0.88/-0.47/0./  -0.47/-0.88/0., %[ 1, 13 ]
```

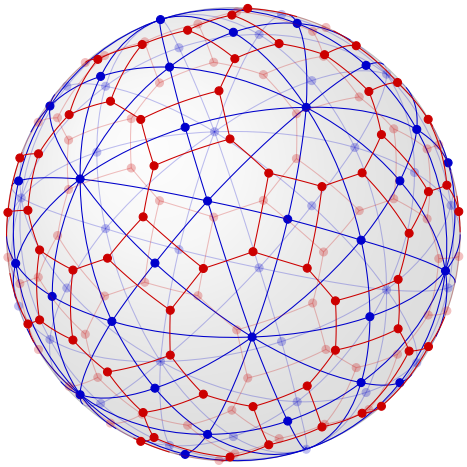
```
0.88/0.47/0./  0.88/0./0.47, %[ 2, 4 ]
0.88/0.47/0./  0.88/0./-0.47, %[ 2, 6 ]
0.88/0.47/0./  0.47/0.88/0., %[ 2, 14 ]
-0.88/0./-0.47/  -0.88/0.47/0., %[ 3, 7 ]
-0.88/0./-0.47/  -0.47/0./-0.88, %[ 3, 9 ]
0.88/0./0.47/  0.88/-0.47/0., %[ 4, 8 ]
0.88/0./0.47/  0.47/0./0.88, %[ 4, 10 ]
-0.88/0./0.47/  -0.88/0.47/0., %[ 5, 7 ]
-0.88/0./0.47/  -0.47/0./0.88, %[ 5, 11 ]
0.88/0./-0.47/  0.88/-0.47/0., %[ 6, 8 ]
0.88/0./-0.47/  0.47/0./-0.88, %[ 6, 12 ]
-0.88/0.47/0./  -0.47/0.88/0., %[ 7, 24 ]
0.88/-0.47/0./  0.47/-0.88/0., %[ 8, 23 ]
-0.47/0./-0.88/  0./-0.47/-0.88, %[ 9, 15 ]
-0.47/0./-0.88/  0./0.47/-0.88, %[ 9, 18 ]
0.47/0./0.88/  0./0.47/0.88, %[ 10, 16 ]
0.47/0./0.88/  0./-0.47/0.88, %[ 10, 17 ]
-0.47/0./0.88/  0./0.47/0.88, %[ 11, 16 ]
-0.47/0./0.88/  0./-0.47/0.88, %[ 11, 17 ]
0.47/0./-0.88/  0./-0.47/-0.88, %[ 12, 15 ]
0.47/0./-0.88/  0./0.47/-0.88, %[ 12, 18 ]
-0.47/-0.88/0./  0./-0.88/-0.47, %[ 13, 19 ]
-0.47/-0.88/0./  0./-0.88/0.47, %[ 13, 21 ]
0.47/0.88/0./  0./0.88/0.47, %[ 14, 20 ]
0.47/0.88/0./  0./0.88/-0.47, %[ 14, 22 ]
0./-0.47/-0.88/  0./-0.88/-0.47, %[ 15, 19 ]
0./0.47/0.88/  0./0.88/0.47, %[ 16, 20 ]
0./-0.47/0.88/  0./-0.88/0.47, %[ 17, 21 ]
0./0.47/-0.88/  0./0.88/-0.47, %[ 18, 22 ]
0./-0.88/-0.47/  0.47/-0.88/0., %[ 19, 23 ]
0./0.88/0.47/  -0.47/0.88/0., %[ 20, 24 ]
0./-0.88/0.47/  0.47/-0.88/0., %[ 21, 23 ]
0./0.88/-0.47/  -0.47/0.88/0. %[ 22, 24 ]
}{
\GCArcABfb{\ax}{\ay}{\az}{\bx}{\by}{\bz}{color=\OMcolor}
}
}
\end{tikzpicture}
```

The preceding example will look as follows



Tope graph picture produced by `LaTeXDrawTopeGraph`.

Both functions `LaTexDrawSpherePicture` and `LaTeXDrawTopeGraph` combined can be used to draw nice pictures of oriented matroid complexes and their duals.



Oriented matroid complex and its dual of $\mathcal{A}(H_3)$.

Chapter 12

Realization spaces of geometric lattices

The computation of the realization space uses the SINGULAR package to call functions from SINGULAR [DGPS24]. Our implementation is an adaptation of the algorithm described in [Cun22, Algorithm 3.9] for geometric lattices of arbitrary rank.

12.1 Construction

12.1.1 LRealizationSpace (for IsGeomLattice, IsInt)

▷ `LRealizationSpace(`*`L, char[, GenSetL]`*`)` (operation)

Returns: RealizationSpaceOfGeometricLattice

Computes the realization space of the geometric lattice *L* in characteristic *char*. Optionally, a generating set of *L* (as a subset of `LAtoms(L)`) can be given. Otherwise, the algorithm tries to find a small one.

Example

```
gap> L:=IntersectionLattice(AGpql(3,3,3));
<Geometric lattice: 9 atoms, rank 3>
gap> RS:=LRealizationSpace(L,0);
<RealizationSpaceOfGeomLattice: in characteristic 0, non-empty: true>
gap> S:=LGenSet(L);
[ 2, 3, 4, 6, 8 ]
gap> RS:=LRealizationSpace(L,0,S);
<RealizationSpaceOfGeomLattice: in characteristic 0, non-empty: true
```

12.2 Attributes

12.2.1 LGenSet (for IsGeomLattice)

▷ `LGenSet(`*`L`*`)` (attribute)

Returns: a set

Computes some generating set of *L*, see [Cun22, Def.3.4] resp. [GMRS26, Def.4.2].

Example

```
gap> L:=IntersectionLattice(AGpql(3,3,3));
<Geometric lattice: 9 atoms, rank 3>
gap> LGenSet(L);
[ 2, 3, 4, 6, 8 ]
```

12.2.2 RSLattice (for IsRealizationSpaceOfGeomLattice)

▷ RSLattice(*RS*) (attribute)

Returns: a geometric lattice

Returns the geometric lattice of the realization space *RS*.

Example

```
gap> RSLattice(RS);
<Geometric lattice: 9 atoms, rank 3>
```

12.2.3 RSCharacteristic (for IsRealizationSpaceOfGeomLattice)

▷ RSCharacteristic(*RS*) (attribute)

Returns: 0 or a prime number

Returns the characteristic of the reliation space *RS*.

Example

```
gap> RSCharacteristic(RS);
0
```

12.2.4 RSDefField (for IsRealizationSpaceOfGeomLattice)

▷ RSDefField(*RS*) (attribute)

Returns: a field

Returns the field over which the realization space *RS* is defined.

Example

```
gap> RSDefField(RS);
Rationals
```

12.2.5 RSCoeffMat (for IsRealizationSpaceOfGeomLattice)

▷ RSCoeffMat(*RS*) (attribute)

Returns: a matrix

Returns the coefficient matrix of *RS* with entries in RSPRing(RS).

Example

```
gap> RSCoeffMat(RS);
[ [ a1-1, a1, 0 ], [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ],
  [ 1, 1, a1 ], [ 1, 1, 1 ], [ 0, 1, -a1^2+a1 ],
  [ 1, 0, a1 ], [ -a1+1, -a1, -a1^2 ] ]
```

12.2.6 RSDimension (for IsRealizationSpaceOfGeomLattice)

▷ RSDimension(*RS*) (attribute)

Returns: a non-negative integer

Returns the dimension of *RS*.

Example

```
gap> RSDimension(RS);
0
```

12.2.7 RSPRing (for IsRealizationSpaceOfGeomLattice)

▷ RSPRing(*RS*) (attribute)

Returns: a polynomial ring

The polynomial ring over which the representing coefficient matrix RSCoeffMat(RS) of *RS* is defined.

Example

```
gap> RSPRing(RS);
Rationals[a1]
```

12.2.8 RSIdealMinors (for IsRealizationSpaceOfGeomLattice)

▷ RSIdealMinors(*RS*) (attribute)

Returns: an ideal

Returns the ideal of equations which have to satisfied due to the dependcy of atoms in RSLattice(RS) over RSDefField(RS).

Example

```
gap> I:=RSIdealMinors(RS); GeneratorsOfIdeal(I);
<two-sided ideal in Rationals[a1], (1 generator)>
[ a1^2-a1+1 ]
```

12.2.9 RSNonMinors (for IsRealizationSpaceOfGeomLattice)

▷ RSNonMinors(*RS*) (attribute)

Returns: a list of polynomials

Gives a list of polynomials which should not vanish due to independent subsets of atoms in RSLattice(RS) over RSDefField(RS).

Example

```
gap> RSNonMinors(RS);
[ a1^3-a1^2, a1^3, -a1^3+2*a1^2-a1, -2*a1^2+2*a1-1 ]
```

12.2.10 RSEvalPoint (for IsRealizationSpaceOfGeomLattice)

▷ RSEvalPoint(*RS*) (attribute)

12.3 Properties

12.3.1 RSIsNonEmpty (for IsRealizationSpaceOfGeomLattice)

▷ RSIsNonEmpty(*RS*) (property)

Returns: true or false

12.4 Further (auxillary) Operations

12.4.1 LSubsetGeneratedByS (for IsGeomLattice,IsList)

▷ LSubsetGeneratedByS(*L*, *S*) (operation)

12.4.2 LIsGenSet (for IsGeomLattice,IsList)

▷ `LIsGenSet(`*L*`, `*S*`)` (operation)

Chapter 13

Orientations of matroids

This chapter describes functions to compute orientations of a given matroid or geometric lattice. For this, an SAT-problem is formulated and solved.

13.1 Orientations of a given geometric lattice

13.1.1 LIsOrientable (for IsGeomLattice)

▷ `LIsOrientable(L)` (property)

Returns: true or false

Determines if there is an oriented matroid with geometric lattice `L`.

Example

13.1.2 LOrientations (for IsGeomLattice)

▷ `LOrientations(L)` (attribute)

Returns: A list of oriented matroids

Returns oriented matroids with geometric lattice `L`.

Example

13.1.3 LFindOrientations (for IsGeomLattice, IsBool)

▷ `LFindOrientations(L, findall)` (operation)

Returns: A list of oriented matroids

Computes oriented matroids (up to OM-isomorphisms) which have `L` as underlying matroid structure or geometric lattice. If `findall` is `true` then all orientations are computed, else at most one orientation is computed.

Depending on the size of `L` this might take a considerable amount of time and there may be many orientations depending on the sparsity of relations in `L`.

The algorithm converts the problem of determining orientations to an SAT-instance and then uses an SAT-solver (currently PicoSAT) to solve.

Example

```
gap> A:=AGpql(2,1,4); L:=IntersectionLattice(A); Bs:=LBases(L);; Length(Bs);
<HyperplaneArrangement: 16 hyperplanes in 4-space>
<Geometric lattice: 16 atoms, rank 4>
```

```
1138
gap> OMs:=LFindOrientations(L,true);
[ <OrientedMatroid: 16 elements, rank 4> ]
```

Chapter 14

SAT-problems

14.1 Construction

14.1.1 SATProblem (for IsInt,IsList, IsBool)

▷ `SATProblem(`*`nvar, clauseslist, allsols`*`)` (operation)

14.2 Attributes

14.2.1 SATnvariables (for IsSATProblem)

▷ `SATnvariables(`*`SProb`*`)` (attribute)

Returns: an integer

Example

14.2.2 SATClauses (for IsSATProblem)

▷ `SATClauses(`*`SProb`*`)` (attribute)

Returns: a list

Example

14.2.3 SATallSolutions (for IsSATProblem)

▷ `SATallSolutions(`*`SProb`*`)` (attribute)

Returns: a list

Example

14.2.4 SATCNFStr (for IsSATProblem)

▷ `SATCNFStr(`*`SProb`*`)` (attribute)

Returns: a string

Example

14.2.5 SATSolutions (for IsSATProblem)

▷ `SATSolutions(`*`SProb`*`)` (attribute)

Returns: a list

`Example`

14.3 Operations

14.3.1 SATSolve (for IsSATProblem)

▷ `SATSolve(`*`SProb, all`*`)` (operation)

Chapter 15

Greedy search for arrangements

This is an implementation of a basic greedy algorithm to construct arrangements satisfying a certain property, based on [Cun22]. A variant was used in [MMR24] to construct an arrangement which is 4-formal with a restriction which is not 3-formal.

15.1 Constructions

15.1.1 HArrGreedySearch (for IsInt,IsInt,IsField,IsFunction,IsInt, IsRat)

▷ HArrGreedySearch(*NumberOfHs, dim, GField, PropTargetFct, MaxNoIterations, tmp*) (operation)

Returns: A hyperplane arrangement

Constructs a greedy search for arrangements over *GField* with *NumberOfHs* hyperplanes in dimension *dim*.

- *PropTargetFct* should be a function taking a hyperplane arragement as argument, returning a non-negative real number, which the search tries to minimize, and such that value=0 means the arrangement satisfies the desired property.
- *MaxNoIterations* is the maximal number of steps to be carried out in the search (one step = exchanging a hyperplane).
- *tmp* is a margin for the target function
- *optional* *StartArr* an arrangement from which to start the search and *FixSubArr* a subarrangement of *StartArr* which should stay fixed during the search.

Example

```
gap> HArrGreedySearch(13,3,GF(17),CharPolySplits,400,0);
GreedySearch over GF(17) for arrangements:
 - of rank 3
 - with 13 hyperplanes.
```

15.1.2 HArrGreedySearchSubArr (for IsRecord)

▷ HArrGreedySearchSubArr(*opts*) (operation)

Returns: A hyperplane arrangement

Constructs a greedy search for arrangements. *opts* should be a record spezifying the following:

- *NumberOfHs* number of hyperplanes,
- *dim* dimension,
- *GField* Galois field,
- *PropTargetFct* should be a function taking a hyperplane arragement as argument, returning a non-negative real number, which the search tries to minimize, and such that value=0 means the arrangement satisfies the desired property.
- *MaxNoIterations* is the maximal number of steps to be carried out in the search (one step = exchanging a hyperplane).
- *tmp* is a margin for the target function.

Example

```
gap> GField:=GF(13);; AFix:=[1..4];;
gap> Astart:=Arr(Concatenation(One(GField)*IdentityMat(3),[List([1..3],x->One(GField))]));;
gap> P:=CharPolySplits;;
gap> opts:=rec(
>     NumberOfHs:=15,
>     dim:=3,
>     GField:=GF(13),
>     PropTargetFct:=P,
>     MaxNoIterations:=1000,
>     tmp := -1/8,
>     StartArr := Astart,
>     FixSubArr := AFix
> );;
gap> GSn:=HArrGreedySearchSubArr(opts); GSRun:=GreedySearchRun(GSn);;
GreedySearch over GF(13) for arrangements:
 - of rank 3
 - with 15 hyperplanes.
gap> A:=GSRun();
260 Iterations - <HyperplaneArrangement: 15 hyperplanes in 3-space>
gap> Print(A);
HyperplaneArrangement defined by
[ [   Z(13)^0,   0*Z(13),   0*Z(13) ],
  [   0*Z(13),   Z(13)^0,   0*Z(13) ],
  [   0*Z(13),   0*Z(13),   Z(13)^0 ],
  [   Z(13)^0,   Z(13)^0,   Z(13)^0 ],
  [   Z(13)^5,   0*Z(13),   Z(13)^0 ],
  [   Z(13)^8,   Z(13)^0,   0*Z(13) ],
  [   Z(13)^8,   0*Z(13),   Z(13)^0 ],
  [   Z(13)^2,   0*Z(13),   Z(13)^0 ],
  [   Z(13)^9,   0*Z(13),   Z(13)^0 ],
  [   Z(13)^9,   Z(13)^0,   0*Z(13) ],
  [   Z(13)^5,   Z(13)^4,   Z(13)^0 ],
  [   Z(13)^0,   0*Z(13),   Z(13)^0 ],
  [   Z(13)^6,   0*Z(13),   Z(13)^0 ],
  [   Z(13)^2,  Z(13)^10,   Z(13)^0 ],
  [   Z(13)^0,   Z(13)^4,   Z(13)^0 ] ]
```

15.1.3 CharPolySplits

▷ CharPolySplits(A) (function)

A target/score function for splitting of the characteristic polynomial of A over $\mathbb{Z}$.

15.1.4 DistMSetInvL

▷ DistMSetInvL(A, B) (function)

A target/score function for comparing MSetInvL(B) and MSetInvL(B) (see MSetInvL (2.2.7)).

15.2 Attributes

15.2.1 GreedySearchRun (for IsHArrGreedySearch)

▷ GreedySearchRun(GS) (attribute)

Returns: a function

Run the search algorithm of GS which will return an arrangement or fail.

Example

```
gap> GS:=HArrGreedySearch(13,3,GF(17),CharPolySplits,400,0);
GreedySearch over GF(17) for arrangements:
 - of rank 3
 - with 13 hyperplanes.
gap> A:=GreedySearchRun(GS)();
227 Iterations - <HyperplaneArrangement: 13 hyperplanes in 3-space>
gap> ExpArr(A);
[ 1, 6, 6 ]
gap> HArrIsSupersolvable(A);
true
gap> A:=GreedySearchRun(GS)();
358 Iterations - <HyperplaneArrangement: 13 hyperplanes in 3-space>
gap> ExpArr(A);
[ 1, 6, 6 ]
gap> HArrIsSupersolvable(A);
false
gap> HArrIsFree(A);
true
```

15.2.2 GreedySearchGF (for IsHArrGreedySearch)

▷ GreedySearchGF(GS) (attribute)

Returns: a field

The Galois field over which to search for arrangements.

Example

15.2.3 GreedySearchDimension (for IsHArrGreedySearch)

▷ GreedySearchDimension(GS) (attribute)

Example

15.2.4 GreedySearchNOfHs (for IsHArrGreedySearch)

▷ `GreedySearchNOfHs(`*GS*`)` (attribute)

Example

15.2.5 GreedySearchTargetFct (for IsHArrGreedySearch)

▷ `GreedySearchTargetFct(`*GS*`)` (attribute)

Example

15.2.6 GreedySearchMaxIterations (for IsHArrGreedySearch)

▷ `GreedySearchMaxIterations(`*GS*`)` (attribute)

Example

References

[ABC+16] T. Abe, M. Barakat, M. Cuntz, T. Hoge, and H. Terao. The freeness of ideal subarrangements of Weyl arrangements. *J. Eur. Math. Soc. (JEMS)*, 18(6):1339–1348, 2016. 25

[Abe16] T. Abe. Divisionally free arrangements of hyperplanes. *Invent. Math.*, 204(1):317–346, 2016. 22

[BLVS+99] Anders Björner, Michel Las Vergnas, Bernd Sturmfels, Neil White, and Günter M. Ziegler. *Oriented matroids*, volume 46 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, Cambridge, second edition, 1999. 4, 15

[BZ92] Anders Björner and Günter M. Ziegler. Combinatorial stratification of complex arrangements. *J. Amer. Math. Soc.*, 5(1):105–149, 1992. 34, 35

[CK22] M. Cuntz and L. Kühne. On arrangements of hyperplanes from connected subgraphs, 2022. 14

[Cun22] Michael Cuntz. A greedy algorithm to compute arrangements of lines in the projective plane. *Discrete Comput. Geom.*, 68(1):107–124, 2022. 49, 57

[DGPS24] Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister, and Hans Schönemann. SINGULAR 4-4-0 — A computer algebra system for polynomial computations. `http://www.singular.uni-kl.de`, 2024. 22, 49

[Ell26] G. Ellis. HAP, Homological Algebra Programming, Version 1.75. `https://gap-packages.github.io/hap`, Mar 2026. GAP package. 36

[Fal95] M. Falk. $K(\pi,1)$ arrangements. *Topology*, 34(1):141–154, 1995. 30

[FR87] M. Falk and R. Randell. On the homotopy theory of arrangements. In *Complex analytic singularities*, volume 8 of *Adv. Stud. Pure Math.*, pages 101–124. North-Holland, Amsterdam, 1987. 29

[GMRS26] L. Giordani, P. Mücksch, G. Röhrle, and J. Schmitt. Hyperpolygonal arrangements. *Glasgow Mathematical Journal*, 68(1):134–146, 2026. 49

[MMR24] Tilman Möller, Paul Mücksch, and Gerhard Röhrle. On formality and combinatorial formality for hyperplane arrangements. *Discrete Comput. Geom.*, 72(1):73–90, 2024. 57

[MY25] P. Mücksch and M. Yoshinaga. Milnor fibrations and oriented matroids. *ArXiv e-prints*, page arXiv:2508.15331, 2025. 36

[OT92] P. Orlik and H. Terao. *Arrangements of Hyperplanes*. Grundlehren der mathematischen Wissenschaften. Springer, 1992. 4

[Sal87] M. Salvetti. Topology of the complement of real hyperplanes in $\mathbf{C}^N$. *Invent. Math.*, 88(3):603–618, 1987. 33

[Ter80] H. Terao. Arrangements of hyperplanes and their freeness I. *J. Fac. Sci. Univ. Tokyo*, 27:293–320, 1980. 22

Index